SWE 537 - Parallel Sorting Report

Roll number: 243203624 Input size: 20000 Algorithm: bubble sort

Recorded program output

serial.c
roll number = 243203624
input size = 20000
serial
execution time = 1.039000034332 seconds
first value = 1
last value = 32767

attempt.c
roll number = 243203624
input size = 20000
attempt
execution time = 0.246999979019 seconds
first value = 3
last value = 32767
sorted correctly = no

parallel.c
roll number = 243203624
input size = 20000
parallel
execution time = 0.095999956131 seconds
first value = 1
last value = 32767
threads = 12

Output comparison

serial out.txt: 20000 values
parallel out.txt: 20000 values
Compare-Object result: Outputs identical.
Therefore, the corrected parallel sequence is identical to the serial sequence.

Parallelization and correction

The serial program performs bubble sort with one thread. The first attempt places an OpenMP parallel-for over dependent bubble-sort passes. Threads can update overlapping array elements at the same time, creating a race condition; the recorded result confirms this with sorted correctly = no.

The corrected parallel program divides the input into four disjoint ranges. OpenMP sections sort the ranges concurrently, so each thread owns separate data and no critical section or lock is needed for swaps. After the sections complete, the ranges are merged serially. This removes the race condition while preserving the selected sorting algorithm.

Speedup and interpretation

Speedup = serial time / parallel time
Speedup = 1.039000034332 / 0.095999956131 = 10.823

The corrected version is faster because independent ranges are sorted concurrently. The attempt is faster than serial in this run but is invalid because its result is incorrect. The recorded machine team had 12 OpenMP threads. Ubuntu lscpu and nproc could not be recorded in this Windows environment because WSL is not installed; available host equivalents were CPU(s) = 12, Core(s) per socket = 6, Thread(s) per core = 2, and nproc equivalent = 12.

Further optimization

A further optimization would parallelize the merge stage as well. It could reduce the remaining serial work, but thread scheduling, memory bandwidth, synchronization, and the small size of individual merge tasks could eliminate the expected benefit on the machine used.